基础通用套件 Java SDK 使用说明

目录

1. ApiClient类介绍
2. UniversalIotKitApi类实例化
3. 快速开始
4. 完整示例
5. 注意事项

ApiClient类介绍

`ApiClient`是所有SDK的核心客户端类，它提供了基于OkHttp的统一的HTTP请求封装、签名自动生成、参数自动加密等功能。它是连接应用程序与大华云开发者平台服务之间的桥梁。

主要功能

1. **HTTP客户端配置**：统一管理请求超时、异常处理等配置
2. **自动签名生成**：通过`SignatureInterceptor`自动为每个请求生成请求头鉴权字段，让开发者不用管理签名鉴权的逻辑，专注于业务逻辑实现。
3. **套件服务创建**：通过`createService()`方法创建各个套件服务接口的实例

构造函数

`ApiClient`提供了多个构造函数以满足不同的使用场景：

```
//通用构造方法
ApiClient apiClient = new ApiClient(String accessKey, String secretKey, String
productId);

// 自定义OkHttpClient.Builder的构造方法
ApiClient apiClient = new ApiClient(String accessKey, String secretKey, String
productId, OkHttpClient.Builder okBuilder);
```

参数说明

- `accessKey`：产品对应的 AccessKey，从产品概况页面获取
- `secretKey`：产品对应的密钥，用于签名生成
- `productId`：产品ID，从产品概况页面获取
- `okBuilder`：OkHttpClient.Builder，当用户想自定义OkHttpClient的时候，传这个参数

UniversalIotKitApi类实例化

使用ApiClient的`createService()`方法创建套件服务接口的实例：

```
// 创建基础通用套件API 服务
UniversalIotKitApi universalIotKitApi =
apiClient.createService(UniversalIotKitApi.class);
```

快速开始

1. 配置基础信息

```
static final String ACCESS_KEY = "19815*******90880";
static final String PRODUCT_ID = "2*******74";
static final String SECRET_KEY = "Brx2*********UAJXbhNOOM";
static final String DEVICE_ID = "AA08*****879";
static final String PASSWORD = "*****";
```

2. 创建 ApiClient

```
ApiClient apiClient = new ApiClient(ACCESS_KEY, SECRET_KEY, PRODUCT_ID);
```

3. 创建服务实例

```
UniversalIotKitApi universalIotKitApi =
apiClient.createService(UniversalIotKitApi.class);
```

4. 获取 AccessToken

```
// 获取 accessToken
 GetAppAccessToken200Response token200Response =
universalIotKitApi.getAppAccessToken();
 GetAppAccessToken200ResponseData data = token200Response.getData();
 String appAccessToken = data.getAppAccessToken();
```

完整示例

设备管理示例

```
import developer.api.core.ApiClient;
import developer.api.model.universalIotKit.request.AddDeviceRequest;
import developer.api.model.universalIotKit.request.DeleteDeviceRequest;
import developer.api.model.universalIotKit.request.GetDeviceListRequest;
import developer.api.model.universalIotKit.response.*;
import developer.api.universalIotKit.UniversalIotKitApi;

import java.io.IOException;
import java.util.List;

public class ApiExample {

    public static void main(String[] args) throws IOException {
```

```
        //产品accessKey
        String accessKey = "198***********0880";
        //产品secretKey
        String secretKey = "XVlfd*****CdYtyAKbd";
        //产品Product ID
        String productId = "22*****574";
        //设备SN
        String deviceId = "AH***HD53";
        //设备品类
        String deviceCategory = "IPC";
        //设备密码（不加密）
        String devCode = "a******n";
        String appAccessToken = null;

        ApiClient apiClient = new ApiClient(accessKey, secretKey, productId);
        UniversalIotKitApi universalIotKitApi =
apiClient.createService(UniversalIotKitApi.class);
        //获取accessToken
        GetAppAccessToken200Response token200Response =
universalIotKitApi.getAppAccessToken();
        GetAppAccessToken200ResponseData data = token200Response.getData();
        appAccessToken = data.getAppAccessToken();
        System.out.println(appAccessToken);

        //添加设备
        AddDeviceRequest addDeviceRequest = new AddDeviceRequest();
        addDeviceRequest.setCategoryCode(deviceCategory);
        addDeviceRequest.setDeviceId(deviceId);
        addDeviceRequest.setDevCode(devCode);
        AddDevice200Response addDeviceResponse =
universalIotKitApi.addDevice(appAccessToken, addDeviceRequest);
        System.out.println(addDeviceResponse.getCode() +
addDeviceResponse.getMsg());

        //查询设备列表
        GetDeviceListRequest request = new GetDeviceListRequest();
        request.setPageNum(1);
        request.setPageSize(10);
        GetDeviceList200Response deviceListResponse =
universalIotKitApi.getDeviceList(appAccessToken, request);
        GetDeviceList200ResponseData deviceListData =
deviceListResponse.getData();
        List<GetDeviceList200ResponseDataPageDataInner> pageData =
deviceListData.getPageData();
        List<GetDeviceList200ResponseDataPageDataInnerDeviceListInner>
deviceList = pageData.get(0).getDeviceList();
        System.out.println(deviceListResponse.getCode() +
deviceListResponse.getMsg() + "设备数量：" + (deviceList == null ? 0 :
deviceList.size()));

        //解绑设备
        DeleteDeviceRequest deleteDeviceRequest = new DeleteDeviceRequest();
        deleteDeviceRequest.setDeviceId(deviceId);
        DeleteDevice200Response deleteDevice200Response =
universalIotKitApi.deleteDevice(appAccessToken, deleteDeviceRequest);
        System.out.println(deleteDevice200Response.getCode() +
deleteDevice200Response.getMsg());
```

```
    }
}
```

注意事项

1. **AccessToken 有效期**：`AppAccessToken` 有时效性，需要定期刷新。建议缓存 token 并在过期前重新获取。
2. **设备密码加密**：使用SDK添加设备时，设备密码不需要加密，方法会自动加密，让开发者不用关心加密算法，专注于业务逻辑实现。。
3. **异常处理**：所有API调用都可能抛出 `RuntimeException`，建议进行适当的异常处理。
4. **响应检查**：调用API后应检查响应的 `success` 字段，确认操作是否成功。

更多信息

- API 接口的详细参数和返回值，开发者平台服务套件文档